tmux documentation

Changelog — vasic-digital/tmux

All releases use Semantic Versioning. Every release carries a positive-runtime-evidence verification record per the project's anti-bluff covenant (Constitution §101 / universal §11.4).

[v1.0.4] — 2026-05-21

CodeGraph code-intelligence integration (§11.4.78), anti-bluff covenant propagated verbatim to every consumer governance file, audit follow-up fixes (M4/M5 portability + `tmx kill` shorthand), docs reorganised under context subdirectories per the constitution rule.

Added

- **A18 — CodeGraph integration (§11.4.78).** Installed `@colbymchenry/codegraph` v0.6.8 globally (npm prefix user-writable; no sudo per §11.4.78). `codegraph init` + indexed; config tracked (`.codegraph/config.json`), DB gitignored (`.codegraph/codegraph.db`). §11.4.10 secret-exclusion patterns + §11.4.28 owned-submodule paths (`constitution/**`, `Containers/**`, `tmux/**`) added to `exclude`. §11.4.77 regeneration mechanism at `.gitignore-meta/codegraph-db.yaml`
    - executable `scripts/codegraph_reindex.sh`. MCP wired for 5 CLI agents:
    - Claude Code (project-scoped `.mcp.json`, NEW)
    - OpenCode (`~/.config/opencode/opencode.json`, pre-existing — audited)
    - Kimi CLI (`~/.kimi/mcp.json`, pre-existing — audited)
    - Crush (`.crush.json`, NEW)
    - Qwen Code (`.qwen/settings.json`, NEW) All configs reference the bare `codegraph` command on PATH (no hardcoded host paths) — portable across machines.
- **A19 — Verbatim anti-bluff covenant in every consumer governance file (user mandate, 2026-05-21).** Inserted the literal 2026-04-28 user-mandate quote into project `CLAUDE.md`, `AGENTS.md`, and `QWEN.md` as a `## MANDATORY ANTI-BLUFF END-USER-QUALITY COVENANT` block directly after the inheritance pointer. Project `Constitution.md` already had it. Tools that don't expand `@imports` now still read the covenant.
- `docs/codegraph/README.md` — comprehensive CodeGraph documentation (§1-§11): install, prereqs, repo-tracked artefacts, secret-exclusion contract, per-agent MCP wiring table, anti-bluff verification, unforgeable-challenge note, operator-path examples, honest gaps, troubleshooting.
- `docs/plans/v1.0.4.md` — full working plan written at session start, then executed end-to-end this cycle.

- `scripts/export_docs.sh` — idempotent §11.4.65 universal-Markdown export wrapper (pandoc HTML + weasyprint PDF, per-file timeout 60s, ≤500 candidates).

Fixed

- **A20 — M4/M5 paired mutations honest topology dispatch (AUDIT-1).** Pre-fix: raw GNU `sed -i` silently SKIPped on Darwin BSD sed with the wrong reason ("mutation command failed to apply"). Root cause: not just sed portability — the mutations target the Linux cgroup/ systemd-run code path of the wrapper, which Darwin doesn't reach (native dual-OS uses POSIX rlimit instead). Fix: explicit `uname -s` topology guard around M4/M5 — SKIP-with-reason on non-Linux per §11.4.3; portable `inplace_sed` on Linux.

- **A21 — `tmx kill` shorthand resolves to `kill-session` (AUDIT-2).** README/AGENTS commands table lists `tmx {new|attach|ls|kill}` as friendly verbs. Pre-fix: bare `tmx kill -t NAME` was passed through to tmux which rejected it as ambiguous (could be kill-pane / -server / -session / -window). Fix: SUBCMD-translation hook in `scripts/tmx.template` detects bare `kill` and rewrites `"$@"` to use `kill-session`.

Hardened (4-layer regression protection per §103)

- **Layer 1 (static gate):** `scripts/verify.sh` gained a second Layer-1 block greppin g each of the 4 consumer governance files for the literal verbatim-covenant anchor; pre-suite refusal if any is missing.
- **Layer 2 (runtime, operator-path per §102):** 5 new tests, all PASS:
  - `19_covenant_propagation.sh` (PASS=7/0/0)
  - `20_codegraph_installed.sh` (PASS=5/0/0)
  - `21_codegraph_index_present.sh` (PASS=4/0/0)
  - `22_codegraph_mcp_wired.sh` (PASS=7/0/0)
  - `23_tmx_kill_shorthand.sh` (PASS=5/0/0)
- **Layer 3 (Challenges):** `TMUX-CH-19` through `TMUX-CH-23` in `scripts/challenges/tmux.yaml`.
- **Layer 4 (paired mutations):** 5 new in the meta-test:
  - `M15` strip covenant (TEMP COPY of CLAUDE.md — real file untouched)
  - `M16` strip `**/*.pem` from `.codegraph/config.json`
  - `M17` strip codegraph from `.mcp.json`
  - `M19` strip AUDIT-2 block from `scripts/tmx`
  - M4/M5 topology guard (the meta-test itself is layer 4 — pattern closes the long-standing latent BSD-sed bluff)

Reorganised

Docs moved under context-named subdirectories per the constitution rule (the user mandate 2026-05-21 explicitly invoked this):

- `docs/GUIDE.md` → `docs/guide/README.md`
- `docs/SCROLLING.md` → `docs/scrolling/README.md`
- `docs/CODEGRAPH.md` → `docs/codegraph/README.md`
- `docs/CONTAINERIZATION_PLAN.md` → `docs/plans/containerization.md`
- `docs/NATIVE_DUAL_OS_PLAN.md` → `docs/plans/native-dual-os.md`
- `docs/PER_SESSION_ISOLATION_PLAN.md` → `docs/plans/per-session-isolation.md`

- `docs/PLAN_v1.0.4.md` → `docs/plans/v1.0.4.md`

Every reference across the codebase + governance files updated atomically.

Verification (this cycle, captured 2026-05-21 on Darwin arm64)

- `bash scripts/setup.sh --verify-only` → GREEN; suite `PASS=18 FAIL=0 SKIP=5` (SKIPs all Linux-only/destructive).
- `bash scripts/tests/meta_test_false_positive_proof.sh` → `26 caught / 0 escaped / 6 skipped` GREEN. The 6 SKIPs are §11.4.3 topology-correct (M4/M5/M7/M8/M9/M10 — Linux-only mutations).
- `bash scripts/test_e2e.sh` → GREEN.
- `bash scripts/codegraph_reindex.sh` → 6 nodes, stamp written.
- §11.4.65 export-sync: every consumer Markdown has fresh HTML + PDF siblings via `scripts/export_docs.sh`.
- §11.4.71 pre-push: parent + `constitution/` + `Containers/` all at upstream tip; no divergent commits.

Out-of-scope this cycle (honest tracking per §11.4.6)

- Upstream `constitution/QWEN.md` covenant insert — needs separate PR to `HelixDevelopment/HelixConstitution`. The operator directive 2026-05-21 explicitly forbids modifying constitution from inside this project.
- `Containers/QWEN.md` create — needs separate PR to `vasic-digital/Containers` (its own §11.4.28 owned-submodule cycle).
- Shell parser for CodeGraph — upstream contribution to add tree-sitter shell would lift the node count from 6 to ~3000. Out of scope per §11.4.74; tracked at the upstream project.
- Agent-driven unforgeable-challenge end-to-end test — classified `AUTONOMOUS_DESIGNED` per §11.4.52 carve-out (mechanical seam exists via test 22 T7; agent-driven layer lands when a headless agent harness is wired).

[v1.0.3] — 2026-05-21

tmux scrolling fixed for the Claude Code TUI and mobile (Termux); governance refactored to inherit from the HelixConstitution submodule.

Fixed

- **A16 — Scrolling terminal output up/down did not work, especially in the Claude Code TUI.** Two root causes: (1) the `history-limit` default (2000) was too small, and (2) tmux's default `WheelUpPane` binding forwards the wheel to applications that request mouse reporting (Claude Code, vim, less) — so the wheel never reached tmux's own scrollback buffer. `scripts/tmux.conf.template` now:
  - bumps `history-limit` to **50000**;
  - sets `mode-keys vi` for vi-style copy-mode navigation;
  - overrides `WheelUpPane` / `WheelDownPane` so the wheel and touch-scroll **always** drive tmux copy-mode scrollback — working identically on a desktop mouse, a trackpad, and a phone (Termux/Android touch-scroll → wheel events);
  - adds the official Claude Code passthrough settings (`allow-passthrough on`, `extended-keys on`, `terminal-features 'xterm*:extkeys'`) so Shift+Enter and escape sequences reach the application;

◦ adds OS-adaptive clipboard routing (pbcopy / wl-copy / xclip / termux-clipboard-set, detected at copy time) plus OSC-52.

Added

- **A17 — HelixConstitution governance submodule + verified inheritance.** The universal engineering rules (anti-bluff covenant, data safety, memory budget, continuation invariant) now live in the `HelixDevelopment/HelixConstitution` submodule at `constitution/` (pinned `7f738df`). The project's `Constitution.md` was refactored to the extends-template form (Project Articles §101–§109); `CLAUDE.md` / `AGENTS.md` gained INHERITED-FROM pointer blocks; a new `QWEN.md` was added for the Qwen Code CLI agent. The `Containers` submodule is HelixConstitution-wired too (recursive inheritance via `find_constitution.sh`).

Hardened (4-layer regression protection per Constitution §103)

- **Layer 1 (static gate):** `scripts/verify.sh` gained a pre-suite static gate that greps `tmux.conf.template` for every scroll setting; RED if any is missing.
- **Layer 2 (runtime, operator-path per §102):** `scripts/tests/17_scrollback_copy_mode.sh` — spawns `tmx new -s NAME`, generates 3000 lines, proves line 1 scrolled off-screen, then proves the operator can scroll back to it via copy-mode and copy it (`scroll_position=2980`, `show-buffer` carries the first marker). PASS=13/0/0. `scripts/tests/18_constitution_inheritance.sh` verifies the submodule + the §11.4 anchor + every project doc's pointer. PASS=10/0/0.
- **Layer 3 (Challenges):** `TMUX-CH-17` and `TMUX-CH-18` in `scripts/challenges/tmux.yaml`.
- **Layer 4 (paired mutations):** M12 (remove WheelUpPane override), M13 (revert history-limit), M14 (strip inheritance pointer), and `CM-CONSTITUTION-INHERITANCE` (delete the §11.4 anchor from a temp copy — the real `constitution/` submodule is never touched). Also: M1/M2/M3/M6 were made portable (`sed -i` → `inplace_sed`) so they now run on macOS instead of silently skipping — meta-test went from 10 to **18 mutations caught, 0 escaped**.

Verification (this cycle, captured 2026-05-21 on Darwin arm64)

- `bash scripts/setup.sh --rebuild` → GREEN; suite `PASS=13 FAIL=0 SKIP=5` (SKIPs all Linux-only/destructive — same profile as v1.0.0).
- `bash scripts/tests/meta_test_false_positive_proof.sh` → `18 caught / 0 escaped / 6 skipped` GREEN.
- `bash scripts/test_e2e.sh` → `PASS=9 FAIL=0 SKIP=0` GREEN.

[v1.0.2] — 2026-05-16

Cosmetic: window-name strips `.exe` suffix from `pane_current_command`.

Fixed

- **A15 — Bottom-left status-bar showed `claude.exe` instead of `claude`** (operator-reported, 2026-05-16). Claude Code v2.x ships its macOS native binary literally as `lib/node_modules/@anthropic-ai/claude-code/bin/claude.exe` (a real Mach-O 64-bit ARM64 executable). The kernel `comm` field carries the on-disk basename, so tmux's `#{pane_current_command}` returned `claude.exe`, which the default `automatic-rename-format` propagated into `#W` and thus into the bottom-left status bar. `scripts/tmux.conf.template` now sets a literal-dot-anchored `.exe` strip in `automatic-rename-format`. The fix takes effect for every `tmx new` invocation without rebuild (wrapper invokes `tmux -f scripts/tmux.conf.template` directly). See `Fixed.md` A15 for the full forensic record.

Hardened (4-layer regression protection per §11.4.4)

- **Layer 1 (static gate):** `scripts/tests/16_window_name_strips_exe.sh` T1 — greps the conf-template for the literal-dot-anchored form.
- **Layer 2 (runtime, operator-path per §11.4.7):** same test, T2/T3 — spawns `tmx new -s NAME`, compiles an in-test `.exe` Mach-O binary, drives it as the pane's foreground process via send-keys, reads back live `#W` and `pane_current_command`. PASS=6 FAIL=0 SKIP=0. Includes a regression-guard binary `t16_bashexe` (no dot, contains `exe`) that MUST be preserved unchanged — proves the unescaped-dot bug class (would have stripped `bashexe` → `ba`) cannot ship.
- **Layer 3 (Challenge):** `TMUX-CH-16` in `scripts/challenges/tmux.yaml`.
- **Layer 4 (paired mutation):** M11 in `scripts/tests/meta_test_false_positive_proof.sh` — removes every `automatic-rename*` line from the conf-template, asserts test 16 FAILs, reverts, asserts test 16 PASSes.

Verified live (positive runtime evidence, this release cycle)

```
# operator-path validation with real claude binary
tmx new -s tmx_live_5198 -d  +  send-keys "exec .../claude"
→ pane_current_command='claude.exe'   #W='claude'
  ✓ defect surface reached (kernel comm reports 'claude.exe')
  ✓ #W stripped to 'claude' (fix doing the work)

# full verify gate
bash scripts/setup.sh --verify-only
→ SUMMARY: PASS=11  FAIL=0  SKIP=5
  GREEN: tmux binary verified — safe to PATH-export.
  (5 SKIPs = pre-existing Linux-only/destructive: 08, 09, 12, 13, 14.)
```

[v1.0.0] — 2026-05-13

Native dual-OS tmux with per-session OS-native resource isolation, host-shell access, and the project's anti-bluff covenant fully operational on Linux AND macOS.

This is the initial public release.

Highlights

- **Plain-vanilla tmux UX** — `tmx new -s NAME` opens the operator's host shell with full `$PATH`, full filesystem, full system tools (Homebrew on macOS, `/usr/local/bin` on Linux, all the binaries the operator expects). No VM, no SSH bridge, no `core@localhost`. The session shell IS the operator's host shell.

- **Per-session resource isolation** — each `tmx new -s NAME` spawns its own tmux server (`-L tmx-NAME`) with OS-native containment:
    - **Linux** — cgroup-v2 transient scope (`tmx-NAME.scope`) via `systemd-run --user --scope`. Kernel-enforced `MemoryMax`, `CPUQuota`, `TasksMax`, `Delegate=yes`. OOM in one scope kills only that scope; `user.slice` survives.
    - **macOS (Darwin)** — POSIX rlimit wrapper applied as session `default-command`. Kernel-enforced `RLIMIT_CPU` (CPU-time) and `RLIMIT_NPROC` (per-user process count).
- **Verified hardened tmux 3.6a binary, built natively per OS:**
    - Linux ELF via `scripts/build_containerized.sh` (podman/docker) or `scripts/build_native.sh`.
    - macOS Mach-O via `scripts/build_native.sh` (Homebrew deps).
    - Compile flags: `-O2 -DNDEBUG -fstack-protector-strong -D_FORTIFY_SOURCE=2`; jemalloc linked at the binary level (`DT_NEEDED` on Linux, `LC_LOAD_DYLIB` on Mach-O); RELRO + bind- at-load (Linux) / `-Wl,-search_paths_first` (Mach-O).
- **Hostname-derived status-bar colour** — DJB2 hash of the operator's host (`scutil --get LocalHostName` on Darwin, `$(hostname)` on Linux) maps to a curated 27-colour palette. Same host always produces the same colour across every session (proven by Test 11); different hosts produce visibly distinct colours (16/16 unique in Test 10 T3).
- **14-test verification gate with positive runtime evidence per §11.4.2** — `scripts/verify.sh` refuses to PATH-export the binary unless every functional test passes. Tests read `/proc/PID/maps`, `/sys/fs/cgroup/.../memory.max`, `tmux show -g status-style`, `tmux capture-pane -p`, and similar live state. Zero tests close on "exit code 0" alone.
- **§11.4.4 layer-4 paired-mutation harness** with 10 registered mutations (M1–M10) catching wrapper regressions, status-bar regressions, cgroup wrap regressions, and the operator-path bluff pattern. `bash scripts/tests/meta_test_false_positive_proof.sh` reports 20 PASS / 0 FAIL on Linux.
- **End-to-end automation** — `bash scripts/test_e2e.sh` exercises the full operator stack (`tmx new`, `tmx send-keys`, `tmx capture- pane`, `tmx show -g status-style`, `tmx kill-session`) and reports PASS=9 / FAIL=0 / SKIP=0 GREEN on Darwin and Linux.
- **OS-aware install** — `bash scripts/setup.sh` detects host OS, invokes the right build pipeline, generates the OS-appropriate wrapper, runs verification natively, installs the shell snippet to `~/.bashrc` AND `~/.zshrc`. Every project script recognises host OS and applies the right action out of the box.

Anti-bluff covenant (Constitution §1, §11.4.x)

This release ships under the verbatim user mandate:

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Every test in `scripts/tests/` carries positive runtime evidence (`/proc`, `/sys/fs/cgroup`, `vmmap`, `ps -o rss=`, `tmux capture- pane`, `systemctl is-active`, `display-message`, kernel log lines). Every Challenge in `scripts/challenges/tmux.yaml` specifies real runtime state as `evidence:`. Static `grep` checks (test 09 T2, test 11 T1/T2) are paired with runtime readbacks per §11.4.7.

Covenant text propagated to root Constitution.md, CLAUDE.md, AGENTS.md, and the Containers submodule's CONSTITUTION.md, CLAUDE.md, AGENTS.md.

Architecture overview

```
                ┌─────────────────────────────────────────┐
                │           OPERATOR SHELL                 │
                │   $ tmx new -s mywork                    │
                │   $ tmx new -s build  ← own scope!       │
                └───────────────────┬─────────────────────┘
                                    │
                                    │  scripts/tmx (host-native, OS-awa
                                    ▼
          ┌─────────────────────────┴───────────────────────────┐
          │                                                     │
     Linux host                                           macOS host (Dar
          │                                                     │
          │  systemd-run --user --scope                         │  tmx-rl
          │    --unit=tmx-NAME.scope                            │    ulim
          │    -p MemoryMax=… CPUQuota=200%                     │    ulim
          │    TasksMax=4096 Delegate=yes                       │    exec
          │    tmux -L tmx-NAME new -s NAME -d                  │
          ▼                                                     ▼
   cgroup-v2 transient scope                              POSIX rlimit wrapp
   (per-group, kernel-enforced)                           (per-process, kern
```

Forensic transparency

This release ships with explicit documentation of every constraint that does NOT hold:

- **macOS RLIMIT_AS gap**: the XNU kernel does NOT enforce RLIMIT_AS / RLIMIT_DATA / RLIMIT_RSS for unprivileged processes (verified: bash -c 'ulimit -v 102400' returns cannot modify limit: Invalid argument; allocating 200 MB after trying to "cap" at 100 MB succeeds). The wrapper applies only what's enforced (RLIMIT_CPU, RLIMIT_NPROC). Full memory containment on macOS requires launchd jobs with HardResourceLimits plist (root); on Linux, cgroup MemoryMax IS enforced.
- **Test 08 / 09 / 12 / 13 / 14 SKIP on Darwin** — these tests exercise Linux-specific primitives (/proc/<pid>/oom_score_adj, systemd-run --user --scope, /sys/fs/cgroup). SKIP-with-reason per Constitution §11.4.3 (per-host-topology test dispatch).

See docs/guide/README.md §5.6 for the full strength-gap table.

Bluffs caught and fixed during the development cycle

Documented in Fixed.md. 24 distinct §1 / §11.4.x bluffs were caught and remediated before this release, including:

- A1: META-MUT-001 paired-mutation harness landed
- A4: Build pipeline three-defect fix (Dockerfile GID-20 collision, jemalloc not actually linked, make-clean missing)

- A6: Install-mechanism side-by-side bluffs (phantom directory path, `alias tmux='tmx'` shadowing system tmux, stale ATMOSphere branding)
- A8: macOS bridge + side-by-side coexistence (later superseded by native dual-OS in this release)
- A10: Status-bar colour silently defaulted to green; bridge ignored macOS hostname
- A11: Triple-layer regression protection so A10 cannot re-occur
- A12: Constitution §11.4.7 — operator-path test coverage rule (every gate test MUST exercise the same entry point an end-user invokes)
- A13: Per-session cgroup isolation (each `tmx new -s X` in its own scope, OOM in one doesn't kill others)
- A14: This-release verification cycle — fresh runtime evidence captured for every claim

Install

macOS (Darwin Apple Silicon or Intel):

```
brew install podman  # optional, for build_containerized.sh
git clone --recurse-submodules git@github.com:vasic-digital/
        tmux.git ~/Projects/tmux
cd ~/Projects/tmux
bash scripts/setup.sh
```

Linux:

```
git clone --recurse-submodules git@github.com:vasic-digital/
        tmux.git ~/Projects/tmux
cd ~/Projects/tmux
sudo bash scripts/install_deps.sh
bash scripts/setup.sh
```

After `setup.sh` reports GREEN: open a new shell (`source ~/.zshrc` or `source ~/.bashrc`). Then `tmx new -s anything` drops you into a session as your host user with the full host environment and kernel-enforced resource caps.

Usage

```
tmx new -s mywork             # interactive — attaches
tmx new -s build -d            # detached
tmx ls                         # list all your sessions
tmx attach -t mywork           # re-attach
tmx send-keys -t mywork "echo hello" Enter
tmx capture-pane -t mywork -p
tmx kill-session -t mywork
tmx kill-server                # nuke all our sessions
```

Per-session resource overrides:

```
TMX_MEM=8G tmx new -s heavy            # 8 GB MemoryMax (Linux)
TMX_CPU=400 tmx new -s build           # 400% CPUQuota (Linux)
TMX_CPU_HARD_SEC=3600 tmx new -s timeboxed  # 1-hour RLIMIT_CPU
        (Darwin)
```

Verification commands

Operators can confirm anti-bluff covenant compliance at any time:

```
bash scripts/setup.sh --verify-only    # full 14-test gate →
        expect GREEN
bash scripts/test_e2e.sh               # end-to-end → expect
        PASS=9/0/0
TMX_TEST_DESTRUCTIVE=1 bash scripts/test_vm.sh   # Linux
        destructive suite
META=1 bash scripts/test_vm.sh         # paired-mutation harness
```

Coexistence with system tmux

`tmx` and the system `tmux` (Homebrew on macOS, distro package on Linux) coexist side-by-side. The bashrc/zshrc snippet PATH-prepends `scripts/` so `tmx` is the project's wrapper; `tmux` resolves to whatever was on PATH before. No alias shadowing.

Submodules

- `tmux/` — upstream `tmux/tmux` pinned to tag `3.6a` (do not modify).
- `Containers/` — `vasic-digital/Containers` Go module providing generic container orchestration primitives. Anti-bluff covenant
    - §11.4.7 propagated. Tag: `b077f2c` at release time.

Known limitations

- macOS memory cap is per-process via launchd-bsd-style limits (informational only — Darwin doesn't enforce `RLIMIT_AS`). For true per-session memory containment, run on Linux.
- The destructive test suite (tests 12 / 13 / 14) requires `TMX_TEST_DESTRUCTIVE=1` and is Linux-only; on Darwin these SKIP per topology dispatch.

Acknowledgements

Built on the shoulders of:

- `tmux/tmux` upstream (3.6a tag)
- jemalloc (linked at build time)
- libevent + ncurses + utf8proc (Mach-O), libtinfo + libevent_core (ELF)
- systemd-run + cgroup-v2 (Linux isolation primitive)
- Homebrew (macOS dependency provider)

[v1.0.1] — Unreleased

Post-release development cycle following the v1.0.0 cut.

- VERSION bumped to `1.0.1` / `versionCode=2` to satisfy the operator's strictly-increasing-version-code mandate immediately after a release.

- `released=` intentionally blank until the next tag is cut (gate value for "not-yet-released"; CI / package builders MUST refuse to publish unless `released=` is populated with the cut date).
- No functional code changes in this entry — this is the post-release bump itself. Subsequent fixes, refactors, and new features will append bullets above this paragraph.